Computer Vision Homework 1

葉騏緯
ID: 114598085

1. Color Transformation: Read a RGB image and write a function to convert the image to grayscale image.

We convert each RGB pixel to a single grayscale intensity by applying the standard luminance-weighted sum.

The mathematical model is

$$Y = 0.299R + 0.587G + 0.114B,$$

which gives higher weight to the green channel because human vision is more sensitive to green brightness changes.

This means the output image keeps the original spatial structure, but each pixel is reduced from three color channels to one intensity channel.

In the actual code, floating-point coefficients are approximated by integers so the conversion can be done efficiently using integer arithmetic:

$$Y \approx \frac{77R + 150G + 29B + 128}{256}.$$

Here, $\frac{77}{256} \approx 0.3008$, $\frac{150}{256} \approx 0.5859$, and $\frac{29}{256} \approx 0.1133$, which closely approximate the BT.601 luminance weights. The extra 128 is added before the right shift so the result is rounded instead of truncated.

The implementation used in the program is the function `rgb_to_gray_value(r, g, b)`, and `main.cpp` calls it through `rik_cv::cvt_color(..., ColorConversion::rgb_to_gray)` for every pixel in the input image.

Code summary:

```
inline std::uint8_t rgb_to_gray_value(std::uint8_t r,
                                      std::uint8_t g,
                                      std::uint8_t b) {
    const std::uint32_t weighted_sum = 77u * r + 150u * g + 29u * b + 128u;
    return static_cast<std::uint8_t>(weighted_sum >> 8);
}
```

Figure 1: Result of RGB-to-grayscale conversion

2. Image Quantization: Reduce the number of intensity levels in a grayscale image. Take the original 256 levels (0-255) and map them to only 4 levels (31, 95, 159, 223).

We model this task as a scalar quantization problem on the grayscale intensity domain. The original 8-bit intensity set $\{0, 1, \ldots, 255\}$ is partitioned into four uniform intervals, and each interval is represented by one reconstruction level.

Let the four quantization intervals be

$$I_0 = [0, 64), \quad I_1 = [64, 128), \quad I_2 = [128, 192), \quad I_3 = [192, 256),$$

and let their reconstruction levels be

$$q_0 = 31, \quad q_1 = 95, \quad q_2 = 159, \quad q_3 = 223.$$

Then the quantizer Q is defined by

$$Q(r) = q_k \quad \text{if } r \in I_k,$$

where r is the input grayscale intensity and $Q(r)$ is the quantized output.

This can be interpreted as replacing each pixel value by the representative value of the interval to which it belongs. Since the four bins all have width 64, the mapping is a uniform 4-level quantizer. The representative values $31, 95, 159, 223$ are the midpoints of the four intervals, so each quantized intensity is the center of its corresponding bin.

The implementation used in the program is `quantize_to_4_levels(value)`, and `main.cpp` applies it to every grayscale pixel through `rik_cv::quantize_4_levels(...)`.

Code summary:

```
inline std::uint8_t quantize_to_4_levels(std::uint8_t value) {
    if (value < 64u) {
        return 31u;
    }
    if (value < 128u) {
        return 95u;
    }
```

```
    if (value < 192u) {
        return 159u;
    }
    return 223u;
}
```

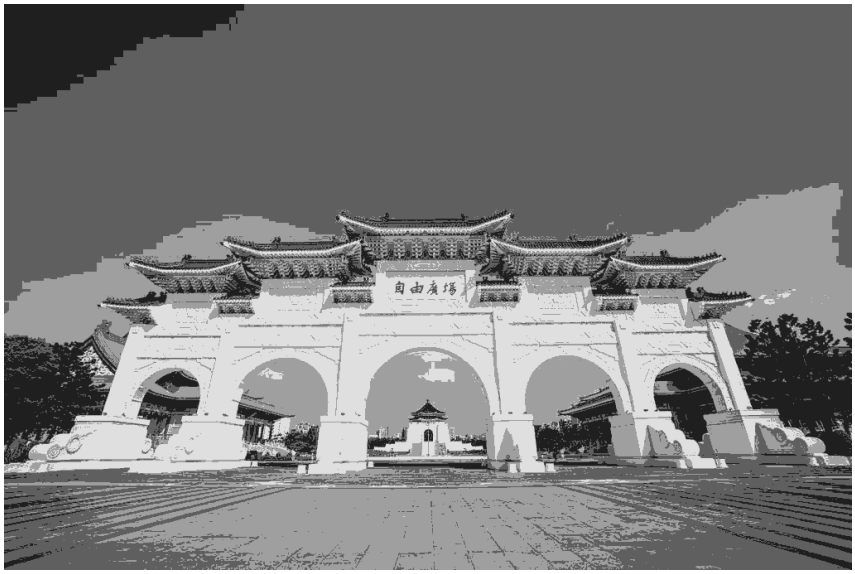


Figure 2: Result of 4-level grayscale quantization

Comparing Fig. 1 and Fig. 2, the overall geometry and major object boundaries remain recognizable after quantization, because only the intensity values are changed while the pixel locations are unchanged. However, Fig. 2 exhibits visibly fewer gray transitions: many smoothly varying regions in Fig. 1 are merged into a small number of flat tonal bands. In other words, the image structure is largely preserved, but the tonal resolution is reduced from 256 levels to 4 levels.

3. Convolution Operation: Using the grayscale/Q1 image, Implement a convolution with 3x3 Box Blur (Average Filter) with padding and stride 1.

$$\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

We model this task as a discrete two-dimensional convolution on the grayscale image. Let the input image be $f(x, y)$ and the blur kernel be $h(i, j)$. The filtered image is defined by

$$g(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} h(i, j) f(x - i, y - j).$$

In this problem, the kernel is

$$h(i, j) = \frac{1}{9} \quad \text{for } (i, j) \in \{-1, 0, 1\} \times \{-1, 0, 1\},$$

so the convolution becomes

$$g(x, y) = \frac{1}{9} \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x - i, y - j).$$

This expression shows that each output pixel is the arithmetic mean of its 3×3 neighborhood. Therefore, the new intensity at (x, y) is no longer determined only by the original pixel itself, but by the local average of nine nearby pixels.

The mathematical reason this kernel causes blur is that averaging suppresses local intensity variation. If a pixel differs sharply from its neighbors, the averaging process pulls it toward the surrounding values. As a result, high-frequency components such as fine texture, sharp edges, and small intensity fluctuations are attenuated, while slowly varying regions are preserved more strongly. In signal-processing terms, the box blur acts as a low-pass filter.

Because the kernel coefficients are all nonnegative and satisfy

$$\sum_{i=-1}^{1} \sum_{j=-1}^{1} h(i, j) = 1,$$

the overall brightness scale is preserved approximately, while local contrast is smoothed.

The implementation is centered on `rik_cv::conv(...)`, whose core logic is written in `project_hw1/include/convolution.hpp`. Instead of only calling a library routine, our code explicitly performs the discrete convolution by scanning the output grid, visiting every kernel position, and accumulating the weighted sum.

Implementation framework:

1. For each output coordinate (x, y), compute the corresponding source origin determined by the stride.
2. For each kernel coordinate (k_x, k_y), shift to the aligned source coordinate and read the corresponding pixel value.
3. If the shifted coordinate lies outside the image domain, the `zero_padding` rule returns intensity 0.
4. Multiply the sampled pixel by the corresponding kernel coefficient and accumulate the result.
5. After the full 3×3 neighborhood has been processed, round and clamp the accumulated value to $[0, 255]$ and write it to the output image.

Code excerpt:

```
for (int y = 0; y < dst.height(); ++y) {
    for (int x = 0; x < dst.width(); ++x) {
        double sum = 0.0;
        const int src_origin_y = y * stride_y;
        const int src_origin_x = x * stride_x;

        for (int ky = 0; ky < kernel_height; ++ky) {
            for (int kx = 0; kx < kernel_width; ++kx) {
                const int src_y = src_origin_y + ky - kernel_anchor_y;
                const int src_x = src_origin_x + kx - kernel_anchor_x;
                const Gray8 sample = std::invoke(padding, src, src_y, src_x);
                sum += static_cast<double>(sample.v) *
                       static_cast<double>(kernel_data[ky * kernel_width + kx]);
            }
        }

        dst_row[x] = Gray8{detail::clamp_to_u8(sum)};
    }
}
```

In `main.cpp`, this operation is invoked by passing the grayscale image, the 3×3 averaging kernel, and stride $(1, 1)$ to `rik_cv::conv(...)`:

```
auto convolved_image =
    rik_cv::conv(gray_view.as_const(), blur_kernel, 3, 3, 1, 1);
```

Figure 3: Result of 3x3 box blur convolution

4. Image Downsampling:
(a) Using the grayscale/Q1 image, implement convolution with 2x2 kernel below using stride 2.
(b) Using the blur/Q3 image, implement convolution with a 2x2 kernel below using stride 2.

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

We use the kernel

$$h = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

together with stride $(2, 2)$.

Since only the upper-left coefficient is nonzero, this kernel does not average neighboring pixels. Instead, from each 2×2 block it keeps only one sample and discards the other three. Mathematically, if the input image is $f(x, y)$, then the output satisfies

$$g(m, n) = f(2m, 2n).$$

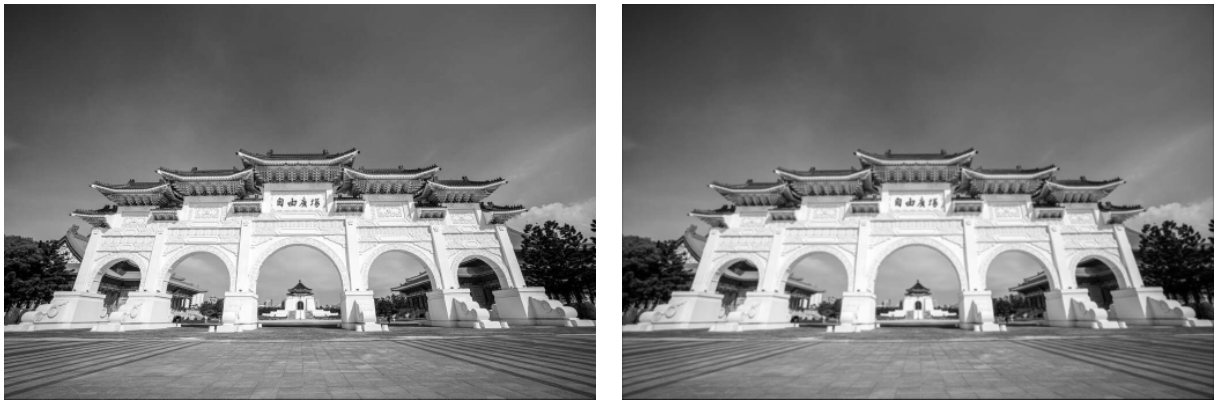
Therefore, the kernel acts as a sampling operator rather than a smoothing operator. Its role is to reduce the spatial resolution by a factor of 2 in both horizontal and vertical directions, while preserving the intensity of the selected sample.

In `main.cpp`, the same kernel is used twice. The result shown in Fig. 4(a) is obtained by applying it directly to the grayscale image from Fig. 1:

```
auto downsampled_gray_image =
    rik_cv::conv(gray_view.as_const(), downsample_kernel, 2, 2, 2, 2);
```

The result shown in Fig. 4(b) applies the same downsampling rule to the blurred image from Fig. 3:

```
auto downsampled_blur_image =
    rik_cv::conv(convolved_image.as_const_view(), downsample_kernel, 2, 2, 2, 2);
```



(a) Downsampling on grayscale image. (b) Downsampling on blurred image.

Figure 4: Comparison of downsampling results. (a) Downsampling on the grayscale image from Fig. 1. (b) Downsampling on the blurred image from Fig. 3.

Comparing Fig. 4(a) and Fig. 4(b), both images have the same reduced spatial resolution because both use the same stride-2 sampling rule. However, Fig. 4(a) is obtained by direct subsampling from the grayscale image, so more local jaggedness and high-frequency detail remain in the sampled result. Fig. 4(b), in contrast, is downsampled after the blur step, so local intensity transitions are smoother and the image appears less noisy and less aliased.